Algoritmos em Grafos - Período - 2026.2
Professor: Carlos Vinicius G. C. Lima

Terceira Lista de Exercícios

Considere o grafo $G = (V, E)$ com $V(G) = \{1, 2, \ldots, n\}$ e os algoritmos de percurso em largura e em profundidade vistos em sala começando no vértice $v \in V(G)$. Considere também G representado por listas de adjacências, a menos que explícito o contrário.

1. Elabore algoritmos para determinar o número de arestas não incidentes a um dado vértice $v \in V(G)$ para as seguintes representações de G e determine a complexidade de cada um.

   (a) Matriz de adjacências.

   (b) Lista de adjacências.

2. Elabore algoritmos para determinar o *complemento* de um grafo simples $G = (V, E)$, ou seja, o grafo $\overline{G} = (V, \overline{E})$) cujos vértices são os mesmos que os de G e o conjunto de arestas é tal que, $uv \in \overline{E}(G)$ se e somente se $uv \notin E(G)$. Considere as seguintes representações de G e determine a complexidade em cada caso.

   (a) Matriz de adjacências.

   (b) Lista de adjacências.

3. Exiba um grafo G com $m \geq 2n$, tal que exista uma execução de Percurso em Largura iniciando em um dado vértice $v \in V(G)$, na qual o tamanho da fila F é máximo logo após a inclusão de um vértice em F cuja distância a v é 3. Indique os estados pelos quais a fila passa na execução que tem a propriedade anterior.

4. Considere o algoritmo PercursoLargura(G, v) que utiliza uma fila F. Demonstre o seguinte lema: seja $u \in V(G)$, tal que $u \neq v$. Quando u é inserido em F, existe um caminho entre v e u em G no qual todos os vértices tem cor **preto**, exceto u. (Dica: por indução no número de execuções de IncluiFila, o algoritmo de inclusão na fila F).

5. Considere G um grafo direcionado sem ciclos orientados (DAG). Demonstre que a ordem inversa em que os vértices são visitados em PercursoProfundidade(G, v) é uma ordenação topológica dos vértices de G.

6. Considere G um grafo direcionado sem ciclos orientados (DAG). Modifique o algoritmo PercursoLargura(G, v) substituindo a fila F por uma pilha P. Seja $\Sigma =< v_1, v_2, \ldots, v_n >$ a sequência em que os vértices são retirados da pilha P em uma execução do algoritmo obtido. Demonstre ou dê um contra-exemplo: Σ define uma ordenação topológica dos vértices de G.

7. Seja $G = (V, E)$ um grafo definido por sua matriz de adjacências $A = V \times V$. Seja $A^k = A \times A \times \cdots \times A$ a matriz obtida pelo produto de k matrizes A, $k \geq 1$. Mostre que se $a_{i,j} \in A^k$ é positivo, então os vértices i e j são conectados por um caminho de comprimento k em G.

8. Seja G um grafo qualquer, modifique o Algoritmo Busca em Profundidade de modo que, dados dois vértices u e v de G, determine se existe um ciclo em G contendo ambos u e v. Determine sua complexidade.

9. Modifique o Algoritmo de Busca em Largura de modo que, dado um grafo conexo e não direcionado G, determine se G é bipartido ou não. A complexidade não pode superar $O(n + m)$.

10. Seja $F =< v_1, v_2, v_3, \ldots, v_\ell >$ o estado da fila F durante no Algoritmo Busca em Largura logo antes a execução da Linha 07 (Enquanto externo). Mostre que ambas as seguintes condições são verdadeiras:

    (a) $d[v_i] \leq d[v_{i+1}]$, para todo $1 \leq i < \ell$.

    (b) $d[v_\ell] \leq d[v_1] + 1$.

    (Dica: use indução no número de vezes que o algoritmo executa a Linha 07).

11. Escreva um algoritmo que determina os caminhos mínimos entre o vértice v e todos os demais vértices de G. Em outras palavras, como podemos determinar os vértices pertencentes aos caminhos mínimos entre v e cada um dos demais vértices, não apenas os valores de distâncias. Seu algoritmo deve ter complexidade $O(n + m)$.